ПР9 Применение отладочных классов в проекте

Часть 1

Сущность объектно-ориентированного подхода к программированию заключается в том, что основные идеи объектно-ориентированного подхода опираются на следующие положения:

- программа представляет собой модель некоторого реального процесса, части реального мира;
- модель реального мира или его части может быть описана как совокупность взаимодействующих между собой объектов;
- объект описывается набором параметров, значения которых определяют состояние объекта, и набором операций (действий), которые может выполнять объект;
- взаимодействие между объектами осуществляется посылкой специальных сообщений от одного объекта к другому. Сообщение, полученное объектом, может потребовать выполнения определенных действий, например, изменения состояния объекта;
- объекты, описанные одним и тем же набором параметров и способные выполнять один и тот же набор действий представляют собой класс однотипных объектов.

С точки зрения языка программирования класс объектов можно рассматривать как тип данного, а отдельный объект - как данное этого типа. Определение программистом собственных классов объектов для конкретного набора задач должно позволить описывать отдельные задачи в терминах самого класса задач (при соответствующем выборе имен типов и имен объектов, их параметров и выполняемых действий).

Таким образом, объектно-ориентированный подход предполагает, что при разработке программы должны быть определены классы используемых в программе объектов и построены их описания, затем созданы экземпляры необходимых объектов и определено взаимодействие между ними.

Классы объектов часто удобно строить так, чтобы они образовывали иерархическую структуру. Например, класс «Студент», описывающий абстрактного студента, может служить основой для построения классов «Студент 1 курса», «Студент 2 курса» и т.д., которые обладают всеми свойствами студента вообще и некоторыми дополнительными свойствами, характеризующими студента конкретного курса.

При разработке интерфейса с пользователем программы могут использовать объекты общего класса «Окно» и объекты классов специальных окон, например, окон информационных сообщений, окон ввода данных и т.п. В таких иерархических структурах один класс может рассматриваться как базовый для других, производных от него классов. Объект производного класса обладает всеми свойствами базового класса и некоторыми собственными свойствами, он может реагировать на те же типы сообщений от других объектов, что и объект базового класса и на сообщения, имеющие смысл только для производного класса. Обычно говорят, что объект производного класса наследует все свойства своего базового класса.

Некоторые параметры объекта могут быть локализованы внутри объекта и недоступны для прямого воздействия извне объекта. Например, во время движения объекта-автомобиля объект-водитель может воздействовать только на ограниченный набор органов управления (рулевое колесо, педали газа, сцепления и тормоза, рычаг

переключения передач) и ему недоступен целый ряд параметров, характеризующих состояние двигателя и автомобиля в целом.

Существует ряд техник, предназначенных для обеспечения качества кода, выполняемых по мере его конструирования.
Основные техники обеспечения качества, используемые в процессе конструирования, включают:
1. рефакторинг;
2. модульное (unit) и интеграционное (integration) тестирование;
3. разработка с первичностью тестов (test-first development - тесты пишутся до конструирования кода);
4. пошаговое кодирование (деятельность по конструированию кода разбивается на мелкие шаги, только после тестирования результатов которых производится переход к следующему шагу кодирования; известен также как итеративное кодирование с тестированием);
5. использование процедур утверждений (assertion);
6. отладка (в привычном понимании — debugging);
7. технические обзоры и оценки (review);
8. статический анализ.

Выбор и использование конкретных техник часто диктуется стандартами (внутренними и внешними), используемыми проектной командой, а также зависят от опыта и подготовленности специалистов, занимающихся конструированием кода.

Ключевым аспектом защитного программирования является использование отладчиков.
Отладчик (debugger) — программа, предназначенная для поиска ошибок в других программах и позволяет выполнять трассировку, отслеживать, устанавливать или изменять значения переменных в процессе выполнения кода, устанавливать и удалять контрольные точки или условия остановки и т. д.

При программировании важно:
1. Не использовать ограничения промышленной версии в отладочной версии.
2. Внедрять поддержку отладки как можно раньше.
3. Использовать наступательное программирование:
- реализовать так, чтобы все утверждения завершали работу программы;
- заполнять всю выделенную память, для обнаружения ошибок выделения памяти;
- заполнять все файлы и потоки;
- при попадании в операторе case в ветви default или else программа прекращает работу;
- заполнять объекты мусором перед их удалением;
- настроить отправку журналов ошибок по электронной почте.
4. Запланировать удаление отладочных средств. Для реализации этого использовать средства автоматизации контроля версий и сборки программ (ant, maven, make), встроенный или собственный препроцессор, отладочные заглушки.

В промышленной версии желательно придерживаться следующих рекомендаций:
1. Оставить код, который проверяет только существенные ошибки.
2. Удалить код, проверяющий незначительные ошибки.
3. Удалить код, приводящий к прекращению работы программы.
4. Оставить код, который позволяет аккуратно завершить программу.
5. Регистрировать ошибки для технической поддержки.
6. Оставленные сообщения об ошибках должны быть дружелюбны.

Задания для выполнения:

1. Создать основной класс. Создать 3 класса наследника. Создать в основном классе 5 методов и 3 свойства, связанных с его назначением, отличных от методов и свойств других подгрупп.
2. Придумать и создать, как минимум, по 2 метода в каждом классе-наследнике (всего не меньше 6) и хотя бы по 2 свойства (всего не меньше 6). Методы и свойства должны быть связаны с особенностями класса-наследника.
3. Сделать, как минимум, 3 метода основного класса переопределёнными в классах наследниках. Сделать, как минимум, 1 метод, который нельзя переопределить.
4. Скомпилировать программу, демонстрирующую работу каждого из методов каждого из классов.

<u>Варианты (по номеру в журнале)</u>

В ходе работы нужно разработать программу.

1,11,21. Основной класс «Автомобили», 3 наследника «Грузовики», «Легковые» и «Автобусы».
2,12,22. Основной класс «Одежда», 3 наследника «Куртки», «Шубы» и «Пуховики».
3,13,23. Основной класс «Еда», 3 наследника «Супы», «Закуски» и «Напитки».
4,14,24. Основной класс «Гаджеты», 3 наследника «Планшеты», «Смартфоны» и «Нетбуки».
5,15,25. Основной класс «Овощи», 3 наследника «Морковь», «Свёкла» и «Лук».
6,16,26. Основной класс «Песни», 3 наследника «Частушки», «Баллады» и «Романсы».
7,17,27. Основной класс «Оружие», 3 наследника «Пистолеты», «Автоматы» и «Пулеметы».
8,18,28. Основной класс «Документы», 3 наследника «Справки», «Приказы» и «Заявления».
9,19,29. Основной класс «Погода», 3 наследника «Пасмурно», «Ясно» и «Ураган».
10,20,30. Основной класс «Звери», 3 наследника «Зайцы», «Волки» и «Лисы».

5. Оформить отчет.

Часть 2.

Отладка (debugging) — это процесс выявления, анализа и устранения ошибок (багов) в программном коде. В .NET-приложениях отладка осуществляется с помощью встроенных инструментов, позволяющих отслеживать выполнение программы, проверять значения переменных и анализировать логику работы.

Важно понимать: **отладочный код не должен присутствовать в финальной (релизной) версии приложения**, так как:

- Замедляет выполнение,
- Увеличивает размер сборки,
- Может раскрывать внутреннюю логику приложения,
- Нарушает принцип "минимализма" релизной сборки.

Для решения этой проблемы в .NET используются **специальные классы и атрибуты**, которые **автоматически исключаются из релизной сборки**.

Классы System.Diagnostics.Debug и System.Diagnostics.Trace

Оба класса предоставляют методы для вывода диагностической информации, но имеют **ключевые различия**:

Характеристика	Debug	Trace
Работает в Debug-режиме	Да	Да
Работает в Release-режиме	Нет	Да (если включены слушатели)
Цель	Временная отладка во время разработки	Диагностика в разработке и продакшене
Компиляция	Исключается при отсутствии символа DEBUG	Не исключается, если не отключены слушатели
Примеры методов	WriteLine(), Assert(), Indent()	WriteLine(), Flush()

По умолчанию Trace не выводит ничего в Release-режиме — нужно явно добавить TraceListener.

Атрибут [Conditional("DEBUG")]

Атрибут [Conditional] применяется к методам и позволяет **условно компилировать** их вызовы в зависимости от наличия определённого символа компиляции.

Пример:

```
[Conditional("DEBUG")]
public void LogInfo(string message)
{
    Console.WriteLine($"[DEBUG] {message}");
}
```

При компиляции в **Debug** — вызов LogInfo("test") включается в IL-код.
При компиляции в **Release** — вызов **полностью удаляется** из сборки (как будто его не было).

Преимущество – нет накладных расходов на вызов метода в Release, даже если метод пустой.
Ограничение – метод должен быть void и не иметь ref/out параметров.

Директивы препроцессора: #if DEBUG, #else, #endif
Это **компиляторные директивы**, позволяющие **условно включать или исключать** фрагменты кода.

Пример:

```
#if DEBUG
    Console.WriteLine("Это сообщение видно только в Debug-режиме");
#endif
```

Если в конфигурации проекта определён символ DEBUG → код включается.
Если DEBUG не определён (например, в Release) → код **полностью игнорируется** компилятором.

Используется для **крупных блоков кода**, например: логирование, тестовые данные, проверки.

Не путать с Debug.WriteLine() — это **другой механизм** (атрибуты и классы), а не препроцессор.

Метод Debug.Assert() — инструмент проверки инвариантов
Debug.Assert(condition, message) — **проверяет условие**.

Если условие false:
- В **Debug-режиме** — программа **останавливается**, открывается окно с сообщением об ошибке.
- В **Release-режиме** — **ничего не происходит** (условие не проверяется).

Пример:
```
Debug.Assert(divisor != 0, "Делитель не может быть нулевым!");
```

Проверка предусловий, постусловий, инвариантов объектов. Отличный способ **поймать ошибку на этапе разработки**, не нарушая работу в продакшене.

Инструменты отладки в Visual Studio 2022

Инструмент	Назначение
Окно Output	Отображает вывод Debug.WriteLine, Trace.WriteLine (выберите "Debug" в выпадающем списке)
Окно Immediate	Позволяет выполнять код в реальном времени во время отладки
Окно Watch	Отслеживает значения переменных
Точка останова (Breakpoint)	Останавливает выполнение на указанной строке
Debug.Assert	Автоматически останавливает выполнение при нарушении условия

Чтобы открыть **Output**: View → Output или Ctrl + Alt + O

При запуске программы можно использовать методы Debug класса для создания сообщений, которые помогают отслеживать последовательность выполнения программы, обнаруживать неисправности или предоставлять сведения о измерении производительности. По умолчанию сообщения, создаваемые Debug классом, отображаются в окне вывода интегрированной среды разработки Visual Studio (IDE).

Методы для работы с классом Debug

Указать сообщение, которое класс создает в качестве первого входного параметра WriteLine метода.

```
Debug.WriteLine("Debug Information-Product Starting ");
```

Для удобства чтения используйте Indent метод для отступа последующих сообщений в окне вывода:

```
Debug.Indent();
```

Чтобы отобразить содержимое выбранных переменных, используйте следующий WriteLine метод:

```
Debug.WriteLine("The product name is " + productName);
```

Можно также использовать WriteLine метод для отображения пространства имен и имени класса для существующего объекта. Например, следующий код отображает System.Xml.XmlDocument пространство имен в окне вывода:

```
System.Xml.XmlDocument oxml = new System.Xml.XmlDocument();
Debug.WriteLine(oxml);
```

Чтобы упорядочить выходные данные, можно включить категорию как необязательный, второй входной параметр WriteLine метода. Если указать категорию, формат сообщения окна вывода будет "категория: сообщение".

```
Debug.WriteLine("The product name is " + productName,"Field");
Debug.WriteLine("The units on hand are" + unitQty,"Field");
Debug.WriteLine("The per unit cost is" + unitCost.ToString(),"Field");
Debug.WriteLine("Total Cost is " + (unitQty * unitCost),"Calc");
```

Окно вывода может отображать сообщения только в том случае, если указанное условие имеет значение true, используя WriteLineIf метод Debug класса. Условие, которое необходимо оценить, является первым входным параметром WriteLineIf метода. Второй параметр WriteLineIf — это сообщение, которое отображается только в том случае, если условие в первом параметре оценивается как true.

```
Debug.WriteLineIf(iUnitQty > 50, "This message WILL appear");
Debug.WriteLineIf(iUnitQty < 50, "This message will NOT appear");
```

Используйте метод Debug Assert класса, чтобы окно вывода отображалось сообщение, только если указанное условие оценивается как false:

```
Debug.Assert(dUnitCost > 1, "Message will NOT appear");
Debug.Assert(dUnitCost < 1, "Message will appear since dUnitcost < 1 is false");
```

Создайте объекты для окна консоли (tr1) и текстового файла с именем *Output.txt* (tr2), а затем добавьте каждый объект в коллекцию ckeifntktq отладки:

```
TextWriterTraceListener tr1 = new TextWriterTraceListener(System.Console.Out);
Debug.Listeners.Add(tr1);
```

```
TextWriterTraceListener tr2 = new TextWriterTraceListener(System.IO.File.CreateText(
"Output.txt"));
Debug.Listeners.Add(tr2);
```

Для удобства чтения используйте Unindent метод, чтобы удалить отступ для последующих сообщений, создаваемых классом Debug . При совместном использовании Indent Unindent методов средство чтения может различать выходные данные как группы.

```
Debug.Unindent();
Debug.WriteLine("Debug Information-Product Ending");
```

Чтобы убедиться, что каждый Listener объект получает все выходные данные, вызовите метод Flush для Debug буферов классов:

```
Debug.Flush();
```

Класс также можно использовать Trace для создания сообщений, отслеживающих выполнение приложения. Debug Классы Trace используют большинство методов для создания выходных данных, включая следующие:

- **WriteLine**
- **WriteLineIf**
- **Отступ**
- **Юниндент**
- **Assert**
- **Краска**

Классы Trace и Debug можно использовать отдельно или вместе в одном приложении.

```
Trace.WriteLine("Trace Information-Product Starting ");
Trace.Indent();

Trace.WriteLine("The product name is " + productName);
Trace.WriteLine("The product name is" + productName, "Field" );
Trace.WriteLineIf(unitQty > 50, "This message WILL appear");
Trace.Assert(unitCost > 1, "Message will NOT appear");

Trace.Unindent();
Trace.WriteLine("Trace Information-Product Ending");

Trace.Flush();
```

Конфигурации сборки: Debug vs Release

Параметр	Debug	Release
Символ компиляции	DEBUG	Нет
Оптимизация кода	Отключена	Включена
Отладочная информация	Full (PDB-файлы)	None или Portable
Производительность	Низкая	Высокая
Отладочные сообщения	Видны	Скрыты (если не Trace)

Как переключиться: В верхней панели Visual Studio → выпадающий список Debug → Release

Когда что использовать?

Ситуация	Рекомендуемый инструмент
Временное логирование во время разработки	Debug.WriteLine()
Проверка инвариантов (например, входные параметры)	Debug.Assert()
Логирование, которое должно работать и в продакшене	Trace.WriteLine() + TraceListener
Включение/выключение больших блоков кода	#if DEBUG
Метод, который должен вызываться только в Debug	[Conditional("DEBUG")]

Ситуация	Рекомендуемый инструмент
Вывод в консоль (всегда)	Console.WriteLine()

Не используйте Console.WriteLine для отладки — он остаётся в релизе и мешает пользователю.

Задание

1. Создание проекта

1) Откройте **Visual Studio**.
2) Создайте новый проект: File → New → Project → выберите **Console App (.NET)**.
3) Назовите проект: **DebugDemo**.
4) Удалите содержимое Program.cs.

2. Реализация класса Calculator с отладкой

Создайте новый файл **Calculator.cs** и вставьте следующий код:

```
using System;
using System.Diagnostics;

namespace DebugDemo
{
    public class Calculator
    {
        /// <summary>
        /// Складывает два числа. Использует Debug и Trace для трассировки.
        /// </summary>
        public double Add(double a, double b)
        {
            Debug.WriteLine($"[DEBUG] Add({a}, {b}) вызван в {DateTime.Now}");
            Debug.Indent();
            Debug.WriteLine($"[DEBUG] Вычисление: {a} + {b}");

            double result = a + b;

            Debug.WriteLine($"[DEBUG] Результат: {result}");
            Debug.Unindent();

            Trace.WriteLine($"[TRACE] Add({a}, {b}) = {result}");

            return result;
        }

        /// <summary>
        /// Делит два числа. Проверяет деление на ноль с помощью Debug.Assert.
        /// </summary>
        public double Divide(double dividend, double divisor)
        {
            #if DEBUG
                Debug.WriteLine($"[DEBUG] Divide({dividend}, {divisor}) — проверка на ноль");
            #endif

            Debug.Assert(divisor != 0, "Попытка деления на ноль! Программа остановлена.");

            if (divisor == 0)
                throw new DivideByZeroException("Деление на ноль запрещено.");

            double result = dividend / divisor;

            // Этот метод вызывается ТОЛЬКО в Debug-режиме
            LogDivision(dividend, divisor, result);

            return result;
        }
```

```
        /// <summary>
        /// Логирует деление — вызывается ТОЛЬКО при определении DEBUG
        /// </summary>
        [Conditional("DEBUG")]
        private void LogDivision(double a, double b, double result)
        {
            Debug.WriteLine($"[DEBUG] Деление: {a} ÷ {b} = {result} (через [Conditional])");
        }

        /// <summary>
        /// Выводит результат в консоль (не отладочный метод)
        /// </summary>
        public void PrintResult(double result)
        {
            Console.WriteLine($"Результат: {result}");
        }
    }
}
```

3. Основная программа Program.cs:

```
using DebugDemo;

class Program
{
    static void Main(string[] args)
    {
        var calc = new Calculator();

        // Тест 1: Сложение
        double result1 = calc.Add(10, 5);
        calc.PrintResult(result1);

        // Тест 2: Деление (нормальное)
        double result2 = calc.Divide(20, 4);
        calc.PrintResult(result2);

        // Тест 3: Деление на ноль — вызовет Debug.Assert в Debug-режиме!
        try
        {
            double result3 = calc.Divide(15, 0);
            calc.PrintResult(result3);
        }
        catch (DivideByZeroException ex)
        {
            Console.WriteLine($"  Ошибка: {ex.Message}");
        }
    }
}
```

4. Запуск

Шаг 1: Запуск в Debug-режиме

1. Убедитесь, что в верхней панели выбрано **Debug**.
2. Нажмите Ctrl + F5 (запуск без отладки).
3. Откройте окно **Output** (Ctrl + Alt + O).
4. В выпадающем списке в Output выберите **Debug**.
5. **Наблюдайте вывод** — должны отображаться:
    - Все Debug.WriteLine
    - Все Trace.WriteLine
    - Окно Debug.Assert при делении на ноль - **нажмите "Break"**, чтобы увидеть стек вызовов.

Шаг 2: Запуск в Release-режиме

1. Переключитесь на **Release** в верхней панели.
2. Нажмите Ctrl + F5.
3. Откройте **Output** → выберите **Debug**.
4. **Что вы видите?**
    - Нет Debug.WriteLine
      Нет [Conditional]-методов
    - Нет #if DEBUG-блоков
    - Есть Trace.WriteLine (если вы не отключили слушатели)
    - Есть Console.WriteLine

Проверьте: В Release-режиме Debug.Assert **не останавливает** программу - только выбрасывается исключение.

5. Включить Trace в Release

Чтобы Trace.WriteLine работал в Release, добавьте в Main():
Trace.Listeners.Add(new ConsoleTraceListener());
Теперь в Release-режиме вы увидите:
[TRACE] Add(10, 5) = 15
[TRACE] Divide(20, 4) = 5

Для продакшена используйте Trace + TextWriterTraceListener для записи в файл.